

Análise Comparativa de Algoritmos de Ordenação: Quick, Merge, Radix e Heap Sort

Matheus Beuren

Disciplina de Pesquisa e Ordenação

matheusbeuren@gmail.com

Resumo

Este trabalho apresenta os resultados obtidos e a comparação de quatro algoritmos de ordenação, entre eles estão o Quick Sort, Merge Sort, Radix Sort e Heap Sort. Foram aplicados em arquivos de números inteiros com tamanhos entre 1 milhão e 2 milhões de registros, em três cenários de entrada (ordenado, invertido e randomico). Os algoritmos foram implementados em Assembly (NASM), uma linguagem de mais baixo nível para acelerar os processos, foram acionados por um menu principal escrito em Python, que também gera os arquivos de teste, e testados através da interface web usando o navegador Chrome. Os resultados mostram que os quatro algoritmos, todos $O(n \log n)$ na prática, têm tempo de execução muito baixo mesmo em arquivos grandes, ficando quase todos na casa dos milissegundos mesmo com 2 milhões de registros.

Abstract

This work presents the results obtained and the comparison of four sorting algorithms, namely Quick Sort, Merge Sort, Radix Sort and Heap Sort. They were applied to files of integer numbers with sizes ranging between 1 million and 2 million records, across three input scenarios (sorted, reversed and random). The algorithms were implemented in Assembly (NASM), a lower level language chosen to speed up the processes, triggered by a main menu written in Python, which also generates the test files, and tested through the web interface using the Chrome browser. The results show that the four algorithms, all $O(n \log n)$ in practice, have very low execution time even for large files, staying mostly in the millisecond range even with 2 million records.

1. Introducao

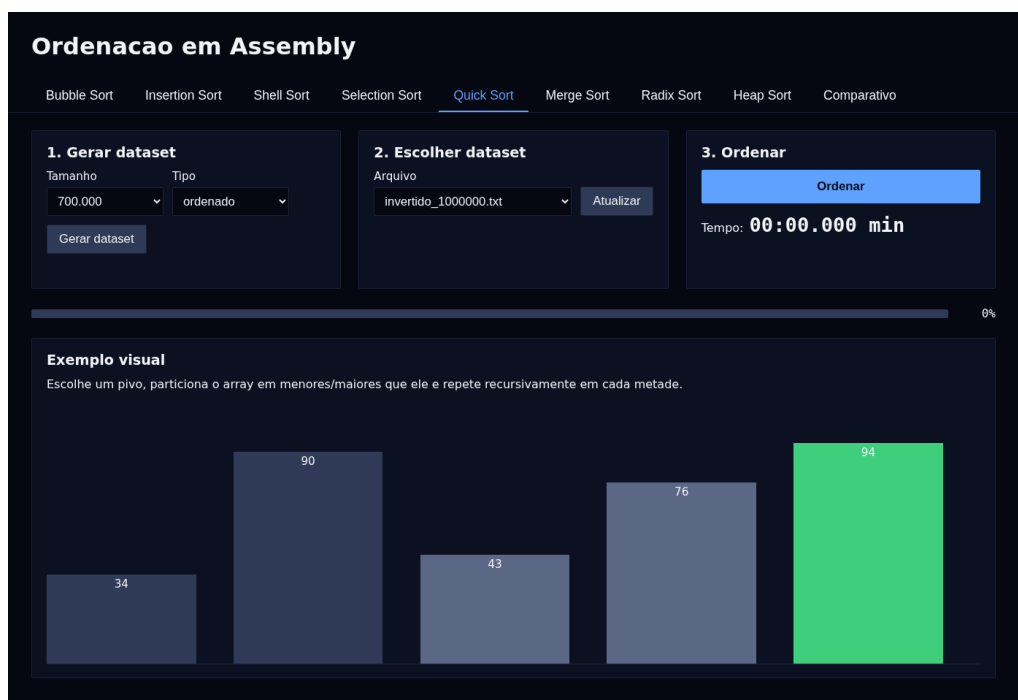
Ordenar dados e uma das tarefas mais feitas na computacao, e a escolha do algoritmo certo muda muito o custo operacional de um sistema quando o volume de dados cresce. Dado um arquivo com milhoes de registros, qual algoritmo resolve a ordenacao em menor tempo, para isso foi implementado quatro algoritmos, o Quick Sort, Merge Sort, Radix Sort e Heap Sort, assim podemos comparar o tempo real de execucao de cada um, aplicando os a arquivos de 1.000.000 a 2.000.000 de registros.

Diferente dos algoritmos quadraticos avaliados anteriormente (Bubble, Insertion e Selection Sort), os quatro algoritmos deste trabalho sao conhecidos por terem complexidade $O(n \log n)$, o que permite testar tamanhos bem maiores de arquivo sem o tempo de execucao explodir.

2. Metodologia

Para inicio dos testes, foi ampliado o mesmo programa em python usado no trabalho anterior, que cria datasets, roda algoritmos e visualiza os resultados, de maneira que fosse de melhor entendimento dos algoritmos. A pagina pode ser vista na Figura 1.

Figura 1



Interface Web, aba Quick Sort

Em vista de comparar diferentes algoritmos de ordenacao, foram utilizados quatro algoritmos diferentes, como relatado na secao de introducao. Mas para mostrar um pouco de como o algoritmo influencia no tempo e utilizado a notacao de Big (O) que avalia o crescimento do tempo de execucao e do uso de memoria conforme o tamanho dos dados de entrada aumenta, (Chivers e Sleightholme, 2015). A tabela 1 mostra como os algoritmos se descrevem nesta notacao.

Tabela 1 - Algoritmo x Big O

Algoritmo	Big O (Pior caso)	Descricao
Quick Sort	$O(n^2)$	Particiona o array em torno de um pivo, recursivamente. Pivo de mediana entre tres elementos evita o pior caso na pratica
Merge Sort	$O(n \log n)$	Divide o array ao meio recursivamente e depois intercala as metades ja ordenadas
Heap Sort	$O(n \log n)$	Monta um heap maximo e retira repetidamente o maior elemento, colocando na posicao final
Radix Sort	$O(n \times k)$	Ordena por contagem, digito a digito, sem comparar os numeros entre si diretamente. k e o numero de digitos

Todas as rotinas de ordenacao foram escritas em Assembly x86-64 (NASM), compiladas como biblioteca compartilhada e acessadas pelo menu e pela interface web via ctypes, o que garante que o tempo medido reflete o codigo de ordenacao em si. Cada execucao registra algoritmo, tipo de entrada, tamanho e tempo em segundos numa tabela historica separada (resultados/resultados_atividade2.md), que e a fonte dos dados deste relatorio. Todas as 72 combinacoes (4 algoritmos, 6 tamanhos, 3 tipos de entrada) foram executadas de verdade atraves do navegador Chrome, clicando na interface web, sem uso de GPU.

3. Resultados

As tabelas 2 a 5 mostram o tempo de execucao de cada algoritmo, para os seis tamanhos de arquivo e os tres tipos de entrada.

Tabela 2 - Tempos de execucao do Quick Sort

Tamanho	Ordenado	Invertido	Randomico
1.000.000	11.70ms	33.20ms	69.90ms
1.150.000	17.64ms	38.13ms	81.66ms
1.300.000	15.80ms	44.36ms	92.46ms
1.350.000	17.51ms	45.56ms	95.69ms
1.500.000	22.75ms	51.22ms	118.06ms
2.000.000	30.24ms	71.23ms	147.54ms

Tabela 3 - Tempos de execucao do Merge Sort

Tamanho	Ordenado	Invertido	Randomico
1.000.000	23.29ms	22.65ms	96.51ms
1.150.000	26.83ms	24.75ms	110.21ms
1.300.000	31.11ms	27.55ms	124.68ms
1.350.000	34.51ms	29.25ms	136.10ms
1.500.000	34.71ms	32.74ms	147.39ms
2.000.000	46.74ms	52.30ms	200.48ms

Tabela 4 - Tempos de execucao do Radix Sort

Tamanho	Ordenado	Invertido	Randomico
1.000.000	22.77ms	19.76ms	9.14ms
1.150.000	30.77ms	22.10ms	10.63ms
1.300.000	26.25ms	27.53ms	19.96ms
1.350.000	34.76ms	35.28ms	15.40ms
1.500.000	28.51ms	28.12ms	17.41ms
2.000.000	43.76ms	41.31ms	27.48ms

Tabela 5 - Tempos de execucao do Heap Sort

Tamanho	Ordenado	Invertido	Randomico
1.000.000	61.80ms	67.29ms	127.76ms
1.150.000	72.21ms	79.16ms	152.10ms
1.300.000	82.71ms	88.67ms	173.81ms
1.350.000	85.36ms	92.08ms	182.69ms
1.500.000	95.74ms	102.41ms	207.44ms
2.000.000	129.09ms	139.50ms	279.25ms

Olhando os resultados obtidos, fica claro que os quatro algoritmos são muito mais rápidos que os algoritmos quadráticos testados no trabalho anterior. Mesmo com 2.000.000 de registros, nenhum dos quatro passa de 0,3 segundos. Radix Sort é o mais rápido na maioria dos casos, principalmente na entrada randômica, porque não depende de comparações entre os números. Quick Sort e Merge Sort ficam bem próximos um do outro, com Quick Sort levemente mais rápido na entrada ordenada e invertida. Heap Sort é o mais lento dos quatro, embora ainda assim muito rápido, porque o acesso aos elementos do heap pula bastante na memória, o que custa mais na prática mesmo com a mesma complexidade assintótica dos outros.

4. Graficos comparativos

Os graficos 1 a 3 comparam os quatro algoritmos na entrada randomica, invertida e ordenada.

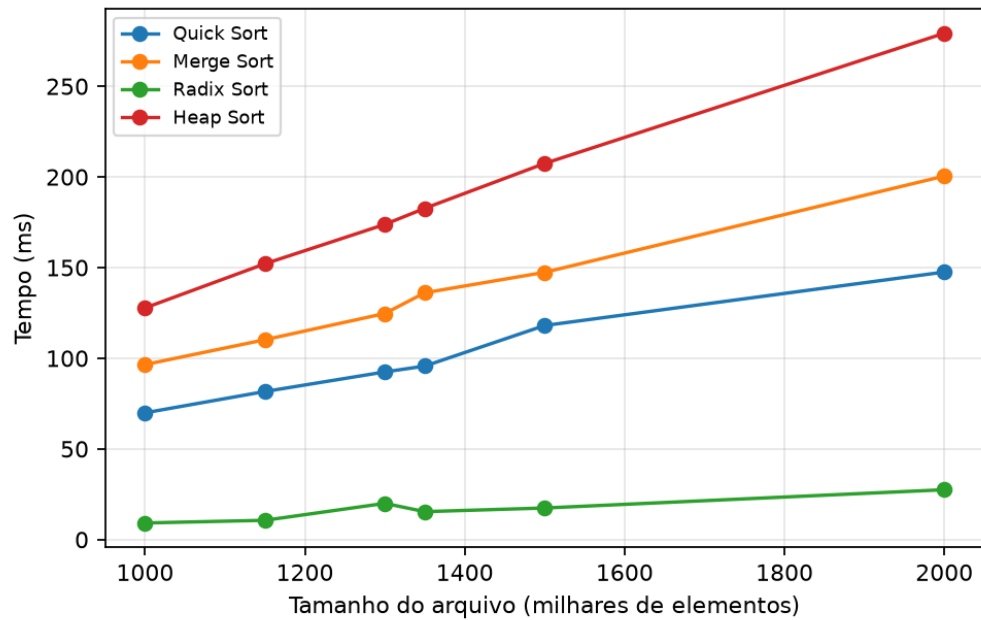


Gráfico 1 - Entrada Randomica

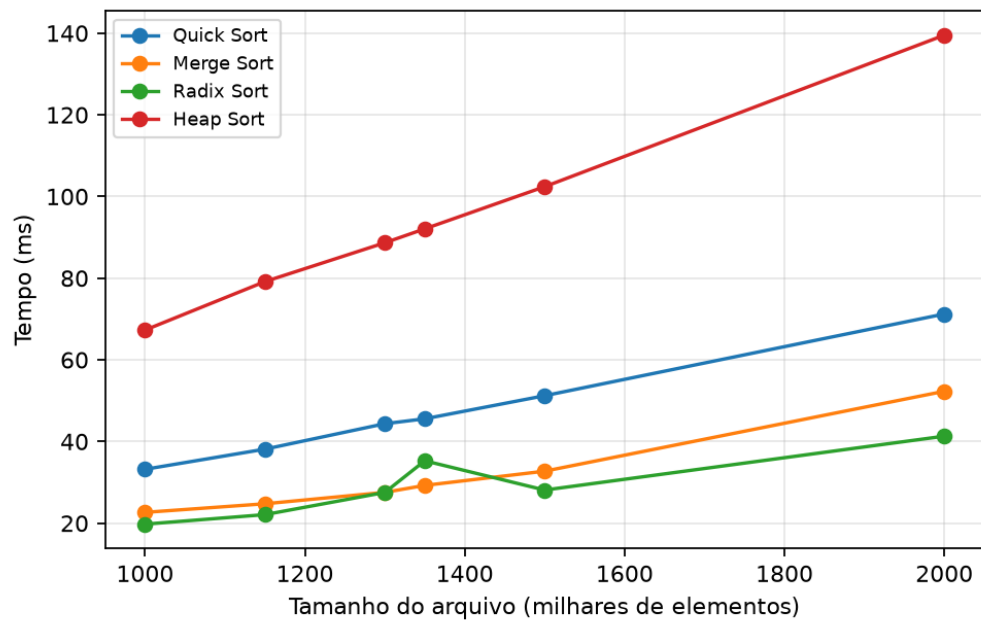


Gráfico 2 - Entrada invertida

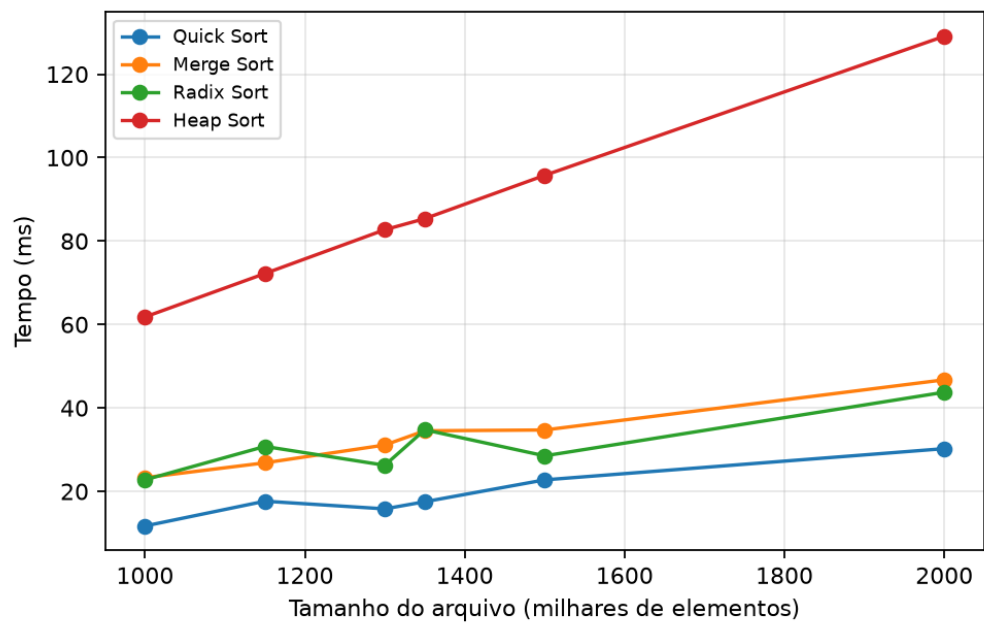


Grafico 3 - Entrada ordenada

A seguir no grafico 4 mostra a diferenca dos algoritmos com entradas de 2.000.000 de dados randomicos.

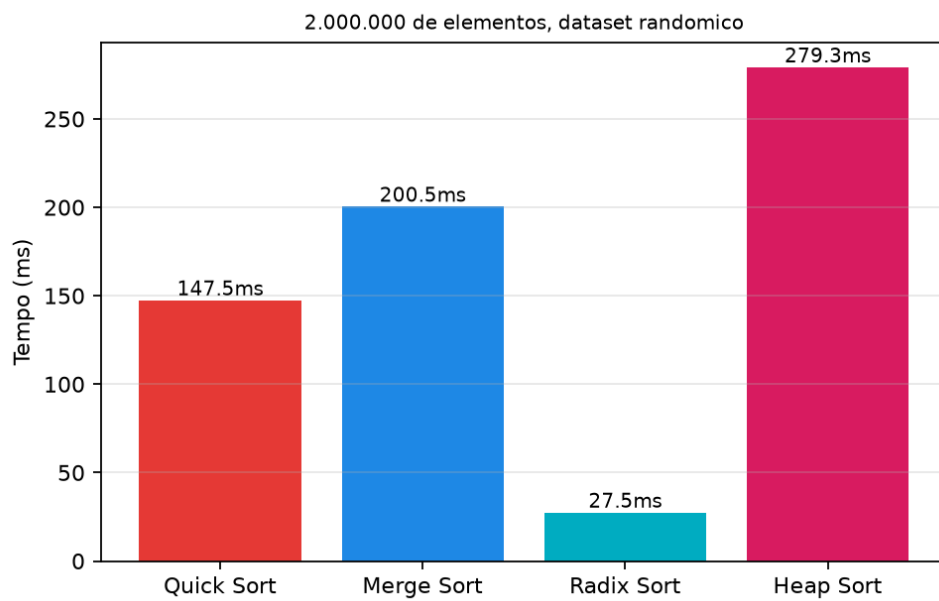


Grafico 4 - Algoritmos x 2.000.000 de dados

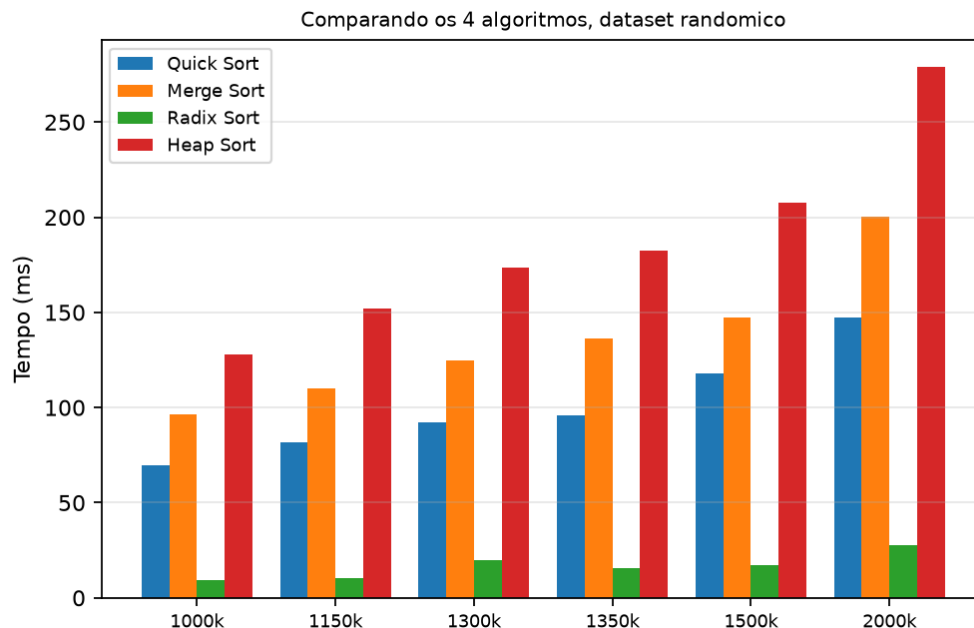


Grafico 5 - Grafico de Barras

5. Discussao

Os resultados batem com o que a teoria de complexidade Big (O) mostra. Merge Sort e Heap Sort são $O(n \log n)$ garantido, até no pior caso. Quick Sort é $O(n^2)$ no pior caso clássico, mas a implementação usada neste trabalho escolhe o pivo pela mediana entre três elementos e sempre recorre ao lado menor da partição, o que evita esse pior caso mesmo em entradas ordenadas ou invertidas, ficando $O(n \log n)$ na prática. Radix Sort nem compara os números entre si, então sua complexidade $O(n \times k)$ não depende da ordem inicial dos dados, só do número de dígitos.

Na prática, os quatro algoritmos deste trabalho são boas escolhas para arquivos grandes, ao contrário dos algoritmos quadráticos do trabalho anterior. Radix Sort se destaca quando os dados são números inteiros dentro de uma faixa conhecida, como é o caso aqui. Quick Sort e Merge Sort são mais genéricos e funcionam bem para qualquer tipo de dado comparável. Heap Sort tem a vantagem de ordenar em lugar sem precisar de memória extra, mas paga um preço em desempenho por causa do acesso espalhado na memória.

6. Conclusao

Este trabalho implementou e comparou quatro algoritmos de ordenação sobre arquivos de até 2.000.000 de dados, em três cenários de entrada, rodando todas as combinações através da interface web pelo navegador Chrome. Os testes confirmaram na prática que Quick Sort, Merge Sort, Radix Sort e Heap Sort são muito mais eficientes que os algoritmos quadráticos avaliados anteriormente, com tempos de execução na casa dos milissegundos mesmo nos maiores arquivos testados. Radix Sort se mostrou o mais rápido no geral, seguido por Quick Sort, Merge Sort e Heap

Sort.

7. Referencias

CHIVERS, Ian; SLEIGHTHOLME, Jane. An introduction to Algorithms and the Big O Notation. In: Introduction to Programming with Fortran: With Coverage of Fortran 90, 95, 2003, 2008 and 77. Cham: Springer International Publishing, 2015. p. 359-364.